Practical 1

Q.no.1-Write programs to parse text files ,CSV,Html, XML and JSON documents and extract relevant data .After retrieving data check any anomalies in the data, missing values etc.

Aim: To write Python programs to parse and extract relevant data from Text, CSV, HTML, XML, and JSON files, and to identify anomalies such as missing values, duplicate records, invalid values, and inconsistent data.

Code:

```
import pandas as pd

import json

import xml.etree.ElementTree as ET

from bs4 import BeautifulSoup

import re

import os

# Common function to check data for anomalies

def check_anomalies(df, name="Data"):

    print(f"\n{'=' * 60}")

    print(f"ANOMALY CHECK: {name}")

    print(f"{'=' * 60}")

    # Missing values

    print("\n1. Missing Values:")

    missing = df.isnull().sum()

    if missing.sum() == 0:

        print("No missing values found.")

    else:

        print(missing[missing > 0])

    # Duplicate rows

    print("\n2. Duplicate Records:")

    duplicates = df.duplicated().sum()

    print("Number of duplicate records:", duplicates)

    # Empty strings

    print("\n3. Empty String Values:")

    empty_values = (df.astype(str).apply(lambda x: x.str.strip() == "").sum())
```

```
    if empty_values.sum() == 0:
        print("No empty string values found.")
    else:
        print(empty_values[empty_values > 0])
    # Numeric columns - negative values
    print("\n4. Negative Numeric Values:")
    numeric_columns = df.select_dtypes(include="number").columns
    for column in numeric_columns:
        negative = (df[column] < 0).sum()
        if negative > 0:
            print(f"{column}: {negative} negative values")
    # Display summary
    print("\n5. Data Summary:")
    print(df.describe(include="all"))



# ------------------------------------------------------------
# 1. Parse TEXT file
# ------------------------------------------------------------
def parse_text_file(filename):
    print("\n\n===== TEXT FILE =====")
    records = []
    with open(filename, "r", encoding="utf-8") as file:
        for line_number, line in enumerate(file, start=1):
            line = line.strip()
            if not line:
                continue
            # Example format:
            # Name: John, Age: 25, City: Mumbai
            fields = {}
            for item in line.split(","):
```

```
            if ":" in item:
                key, value = item.split(":", 1)
                fields[key.strip()] = value.strip()
        if fields:
            records.append(fields)
    df = pd.DataFrame(records)
    print("\nExtracted Data:")
    print(df)
    check_anomalies(df, "Text File")
    return df
# ------------------------------------------------------------
# 2. Parse CSV file
# ------------------------------------------------------------
def parse_csv_file(filename):
    print("\n\n===== CSV FILE =====")
    try:
        df = pd.read_csv(filename)
        print("\nExtracted Data:")
        print(df)
        check_anomalies(df, "CSV File")
        return df
    except Exception as e:
        print("Error reading CSV:", e)
        return None
```

```
# ------------------------------------------------------------
# 3. Parse HTML file
# ------------------------------------------------------------
def parse_html_file(filename):
    print("\n\n===== HTML FILE =====")
    with open(filename, "r", encoding="utf-8") as file:
        html = file.read()
    soup = BeautifulSoup(html, "html.parser")
    records = []
    # Extract HTML tables
    tables = soup.find_all("table")
    for table in tables:
        rows = table.find_all("tr")
        headers = []
        for row_index, row in enumerate(rows):
            cells = row.find_all(["th", "td"])
            values = [cell.get_text(strip=True) for cell in cells]
            if row_index == 0:
                headers = values
            else:
                if values:
                    if headers and len(values) == len(headers):
                        records.append(dict(zip(headers, values)))
    if records:
        df = pd.DataFrame(records)

        print("\nExtracted Data:")
        print(df)
        check_anomalies(df, "HTML File")
```

```
    return df
print("No table data found in HTML file.")
return None
```

Output:

```
DATA PARSING AND ANOMALY DETECTION PROGRAM

    Select file type:

    1. Text
    2. CSV
    3. HTML
    4. XML
    5. JSON

Enter your choice:  2
Enter file name/path:  student_performance_dataset.csv


===== CSV FILE =====

Extracted Data:
     student_id  gender  study_time_hours  attendance_percent  sleep_hours  \
0             1    Male               4.0                98.0          6.5
1             2  Female               6.3               100.0          5.7
2             3    Male               4.9                85.3          7.9
3             4    Male               2.6                77.5          8.0
4             5    Male               2.2                89.6          4.6
..          ...     ...               ...                 ...          ...
995         996    Male               2.8                75.4          8.2
996         997    Male               6.7                88.4          7.1
997         998  Female               2.6                84.5          8.0

     parental_education internet_access extracurricular_activities  \
0             Bachelors             Yes                        Yes
1           High School             Yes                        Yes
2             Bachelors             Yes                         No
3                   NaN             Yes                        Yes
4             Bachelors             Yes                         No
..                  ...             ...                        ...
995             Masters             Yes                        Yes
996                 NaN              No                        Yes
997         High School             Yes                        Yes
998         High School              No                         No
999             Masters             Yes                         No

     part_time_job  previous_grade  final_exam_score final_grade
0               No            76.9             100.0           A
1              Yes            75.5             100.0           A
2              Yes            88.5              97.3           A
3               No            85.1              83.8           B
4              Yes            61.8              68.3           D
..             ...             ...               ...         ...
995             No            60.5              70.7           C
996             No            82.2              99.5           A
997            Yes            65.2              79.2           C
998            Yes            52.2              82.2           B
999             No            45.4              70.4           C

[1000 rows x 12 columns]

============================================================
ANOMALY CHECK: CSV File
```

```
============================================================
ANOMALY CHECK: CSV File
============================================================

1. Missing Values:
parental_education    102
dtype: int64

2. Duplicate Records:
Number of duplicate records: 0

3. Empty String Values:
No empty string values found.

4. Negative Numeric Values:

5. Data Summary:
         student_id  gender  study_time_hours  attendance_percent  \
count   1000.000000    1000       1000.000000         1000.000000
unique          NaN       2               NaN                 NaN
top             NaN  Female               NaN                 NaN
freq            NaN     510               NaN                 NaN
mean     500.500000     NaN          3.570700           85.092300
std      288.819436     NaN          1.478559            9.270685
min        1.000000     NaN          0.500000           54.800000
25%      250.750000     NaN          2.600000           78.800000
50%      500.500000     NaN          3.600000           85.200000
75%      750.250000     NaN          4.500000           91.900000
max     1000.000000     NaN          8.100000          100.000000
```

	extracurricular_activities	part_time_job	previous_grade	\
count	1000	1000	1000.000000	
unique	2	2	NaN	
top	Yes	No	NaN	
freq	572	684	NaN	
mean	NaN	NaN	69.740900	
std	NaN	NaN	12.613425	
min	NaN	NaN	31.300000	
25%	NaN	NaN	61.000000	
50%	NaN	NaN	69.600000	
75%	NaN	NaN	78.400000	
max	NaN	NaN	100.000000	

	final_exam_score	final_grade
count	1000.000000	1000
unique	NaN	5
top	NaN	B
freq	NaN	354
mean	83.543500	NaN
std	10.341333	NaN
min	46.800000	NaN
25%	76.075000	NaN
50%	83.800000	NaN
75%	91.525000	NaN
max	100.000000	NaN

```
===== DATA VALIDATION =====

Required Column Check:
Name: MISSING
Age: MISSING
Email: MISSING

Validation completed.
```

Q.no.2-Write programs for reading and writing binary files.

Aim:To write Python programs for reading data from and writing data to binary files using file handling operations.

Program: Read and Write a Binary File

```
# Writing data to a binary file
data = b"Hello, this is binary file data."
with open("sample.bin", "wb") as file:
    file.write(data)
print("Data written successfully.")
# Reading data from the binary file
with open("sample.bin", "rb") as file:
    data = file.read()
print("Data read from binary file:")
print(data)
# Convert bytes to string
print("Decoded data:")
print(data.decode())
```

Output:

```
Data written successfully.
Data read from binary file:
b'Hello, this is binary file data.'
Decoded data:
Hello, this is binary file data.
```

Q.no.3- Write prog for searching ,splitting , and replacing strings based on pattern matching using regular expression

Aim

To write a Python program for searching, splitting, and replacing strings based on pattern matching using regular expressions.

Program

```
import re
text = "My email is john@gmail.com and my phone number is 9876543210."
# 1. Searching using regular expression
pattern = r"\b[\w.-]+@[\w.-]+\.\w+\b"
match = re.search(pattern, text)
if match:
    print("Email found:", match.group())
else:
```

```
    print("Email not found")
# 2. Splitting using regular expression
sentence = "Python,Java;C++:HTML"
result = re.split(r"[,;:]", sentence)
print("\nAfter splitting:")
print(result)
# 3. Replacing using regular expression
new_text = re.sub(r"\d", "*", text)
print("\nAfter replacing digits:")
print(new_text)



# ------------------------------------------------------------
# 4. Parse XML file
# ------------------------------------------------------------
def parse_xml_file(filename):
    print("\n\n===== XML FILE =====")
    tree = ET.parse(filename)
    root = tree.getroot()
    records = []
    # Assumes XML contains repeated child elements such as:
    # <students>
    #       <name>John</name>
    #       <age>20</age>
    #    </student>
    # </students>
    for item in root:
        record = {}
        for child in item:
            record[child.tag] = child.text
        if record:
            records.append(record)
```

```
    df = pd.DataFrame(records)
    print("\nExtracted Data:")
    print(df)
    check_anomalies(df, "XML File")
    return df
# ------------------------------------------------------------
# 5. Parse JSON file
# ------------------------------------------------------------
def parse_json_file(filename):
    print("\n\n===== JSON FILE =====")
    with open(filename, "r", encoding="utf-8") as file:
        data = json.load(file)
    # JSON may contain either:
    # [
    #   {"name": "John", "age": 20},
    #   {"name": "Alice", "age": 22}
    # ]
    #
    # or:
    # {"students": [...]}
    if isinstance(data, dict):
        # Find the first list inside the JSON object
        list_data = None
        for value in data.values():
            if isinstance(value, list):
                list_data = value
                break
        if list_data is not None:
            data = list_data
        else:
            data = [data]
    df = pd.json_normalize(data)
```

```
    print("\nExtracted Data:")
    print(df)
    check_anomalies(df, "JSON File")
    return df
# ------------------------------------------------------------
# Additional validation
# ------------------------------------------------------------
def validate_data(df):
    print("\n\n===== DATA VALIDATION =====")
    # Check required columns
    required_columns = ["Name", "Age", "Email"]
    print("\nRequired Column Check:")
    for column in required_columns:
        if column in df.columns:
            print(f"{column}: Found")
        else:
            print(f"{column}: MISSING")
    # Check email format
    if "Email" in df.columns:
        print("\nInvalid Email Addresses:")
        email_pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"
        for index, email in df["Email"].items():
            if pd.notna(email):
                if not re.match(email_pattern, str(email)):
                    print(f"Row {index}: {email}")
    # Check age
    if "Age" in df.columns:
        print("\nInvalid Age Values:")
        for index, age in df["Age"].items():
            try:
                age = float(age)
                if age < 0 or age > 120:
```

```
                print(f"Row {index}: Invalid age = {age}")
            except (ValueError, TypeError):
                print(f"Row {index}: Non-numeric age = {age}")

    print("\nValidation completed.")
# ------------------------------------------------------------
# Main program
# ------------------------------------------------------------
def main():
    print("DATA PARSING AND ANOMALY DETECTION PROGRAM")
    print("""
    Select file type:
    1. Text
    2. CSV
    3. HTML
    4. XML
    5. JSON
    """)
    choice = input("Enter your choice: ")
    filename = input("Enter file name/path: ")
    if not os.path.exists(filename):
        print("File does not exist.")
        return
    if choice == "1":
        df = parse_text_file(filename)
    elif choice == "2":
        df = parse_csv_file(filename)
    elif choice == "3":
        df = parse_html_file(filename)
    elif choice == "4":
        df = parse_xml_file(filename)
    elif choice == "5":
```

```
        df = parse_json_file(filename)
    else:
        print("Invalid choice.")
        return
    if df is not None and not df.empty:
        validate_data(df)
if __name__ == "__main__":
    main()
```

Output:

```
Email found: john@gmail.com

After splitting:
['Python', 'Java', 'C++', 'HTML']

After replacing digits:
My email is john@gmail.com and my phone number is **********.
```

Q.no.4- Design a relational database for a small app and populate the database. Using sql do the CRUD (create , read, updated and delete) operation.

Practical 2

Aim

To implement an ETL workflow using SQLite by extracting raw product data, transforming it into processed product data, calculating total sales, and performing category-wise and city-wise sales analysis.

Code

```
import sqlite3
import os

# Define the database file name
db_file = 'BI_ETL_Practical2.db'

cnxn = None
try:
    # Connect to SQLite database (creates the file if it doesn't exist)
    cnxn = sqlite3.connect(db_file)
    cursor = cnxn.cursor()
    print(f"Successfully connected to SQLite database: {db_file}")

    # Optionally, clear existing tables for a fresh run if the file already exists
    # (Uncomment if you want to always start with a clean slate)
    # cursor.execute("DROP TABLE IF EXISTS Processed_Product_Data;")
    # cursor.execute("DROP TABLE IF EXISTS Raw_Product_Data;")
    # cnxn.commit()
    # print("Existing tables dropped (if any).")

except sqlite3.Error as ex:
    print(f"Error connecting to SQLite database: {ex}")
    cnxn = None

# Ensure connection is established before proceeding
if cnxn is None:
    raise Exception("Failed to establish database connection.")
```

Output

```
Successfully connected to SQLite database: BI_ETL_Practical2.db
```

```
if cnxn:
    cursor = cnxn.cursor()
    try:
        # Step 2: Create Staging Table
        cursor.execute("""
        CREATE TABLE IF NOT EXISTS Raw_Product_Data
        (
            ProductID INTEGER,
            ProductName VARCHAR(100),
            Category VARCHAR(50),
            Quantity INTEGER,
            Price DECIMAL(10,2),
            SaleDate VARCHAR(20),
            City VARCHAR(50)
        );
        """)
        print("Raw_Product_Data table created or already exists.")

        # Step 3: Extract / Insert Raw Data
        # Check if data already exists to prevent duplicate inserts on re-run
        cursor.execute("SELECT COUNT(*) FROM Raw_Product_Data;")
        if cursor.fetchone()[0] == 0:
            cursor.execute("""
            INSERT INTO Raw_Product_Data
            (ProductID, ProductName, Category, Quantity, Price, SaleDate, City)
            VALUES
            (101, 'Laptop', 'Electronics', 2, 55000, '2026-01-10', 'Mumbai'),
            (102, 'Mobile', 'Electronics', 3, 25000, '2026-01-12', 'Pune'),
            (103, 'Chair', 'Furniture', 5, 4500, '2026-01-15', 'Nashik'),
            (104, 'Table', 'Furniture', 2, 8000, '2026-01-18', 'Mumbai'),
            (105, 'Headphones', 'Electronics', 4, 2000, '2026-01-20', 'Nagpur');
            """)
            print("Data inserted into Raw_Product_Data.")
        else:
            print("Raw_Product_Data already contains data. Skipping insert.")
```

```
# Step 4: Display Raw Data (using fetchall() and pandas for better display)
cursor.execute("SELECT * FROM Raw_Product_Data;")
raw_data = cursor.fetchall()
import pandas as pd
raw_df = pd.DataFrame.from_records(raw_data, columns=[desc[0] for desc in cursor.description])
print("\nRaw Product Data:")
display(raw_df)

# Step 5: Create Final Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Processed_Product_Data
(
    ProductID INTEGER PRIMARY KEY,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    Quantity INTEGER,
    Price DECIMAL(10,2),
    SaleDate DATE,
    City VARCHAR(50),
    TotalAmount DECIMAL(12,2)
);
""")
print("Processed_Product_Data table created or already exists.")

# Step 6: Transform and Load Data
# The date is converted from VARCHAR to DATE, and the total amount is calculated.
# Check if data already exists to prevent duplicate inserts on re-run
cursor.execute("SELECT COUNT(*) FROM Processed_Product_Data;")
if cursor.fetchone()[0] == 0:
    cursor.execute("""
    INSERT INTO Processed_Product_Data
    (
        ProductID,
        ProductName,
        Category,
        Quantity,
        Price,
        SaleDate,
        City,
        TotalAmount
    )
    SELECT
        ProductID,
        ProductName,
        Category,
        Quantity,
        Price,
        DATE(SaleDate), -- Changed CONVERT(DATE, SaleDate) to DATE(SaleDate) for SQLite
        City,
        Quantity * Price
    FROM Raw_Product_Data;
    """)
    print("Data transformed and loaded into Processed_Product_Data.")
else:
    print("Processed_Product_Data already contains data. Skipping transform and load.")

# Step 7: Display Final Transformed Data
cursor.execute("SELECT * FROM Processed_Product_Data;")
processed_data = cursor.fetchall()
processed_df = pd.DataFrame.from_records(processed_data, columns=[desc[0] for desc in cursor.description])
print("\nProcessed Product Data:")
display(processed_df)

# Step 8: Perform Analysis
print("\nPerforming Analysis:")
# Total Sales
cursor.execute("SELECT SUM(TotalAmount) AS Total_Sales FROM Processed_Product_Data;")
total_sales = cursor.fetchone()[0]
print(f"Total Sales: {total_sales}")

# Category-wise Sales
cursor.execute("""
SELECT
    Category,
```

```
            SUM(TotalAmount) AS Category_Total
        FROM Processed_Product_Data
        GROUP BY Category;
        """)
        category_sales = cursor.fetchall()
        category_sales_df = pd.DataFrame.from_records(category_sales, columns=[desc[0] for desc in
cursor.description])
        print("\nCategory-wise Sales:")
        display(category_sales_df)

        # City-wise Sales
        cursor.execute("""
        SELECT
            City,
            SUM(TotalAmount) AS City_Total
        FROM Processed_Product_Data
        GROUP BY City;
        """)
        city_sales = cursor.fetchall()
        city_sales_df = pd.DataFrame.from_records(city_sales, columns=[desc[0] for desc in cursor.description])
        print("\nCity-wise Sales:")
        display(city_sales_df)

        cnxn.commit() # Commit all transactions
        print("All SQL operations completed successfully and committed.")

    except sqlite3.Error as ex:
        print(f"Error during SQL execution: {ex}")
        cnxn.rollback() # Rollback in case of error
        print("Transaction rolled back.")
    finally:
        cursor.close()
        cnxn.close()
        print("Database connection closed.")
else:
    print("No active database connection to execute SQL queries.")
```

Output

```
Raw_Product_Data table created or already exists.
Data inserted into Raw_Product_Data.

Raw Product Data:

Processed_Product_Data table created or already exists.
Data transformed and loaded into Processed_Product_Data.

Processed Product Data:


Performing Analysis:
Total Sales: 231500

Category-wise Sales:

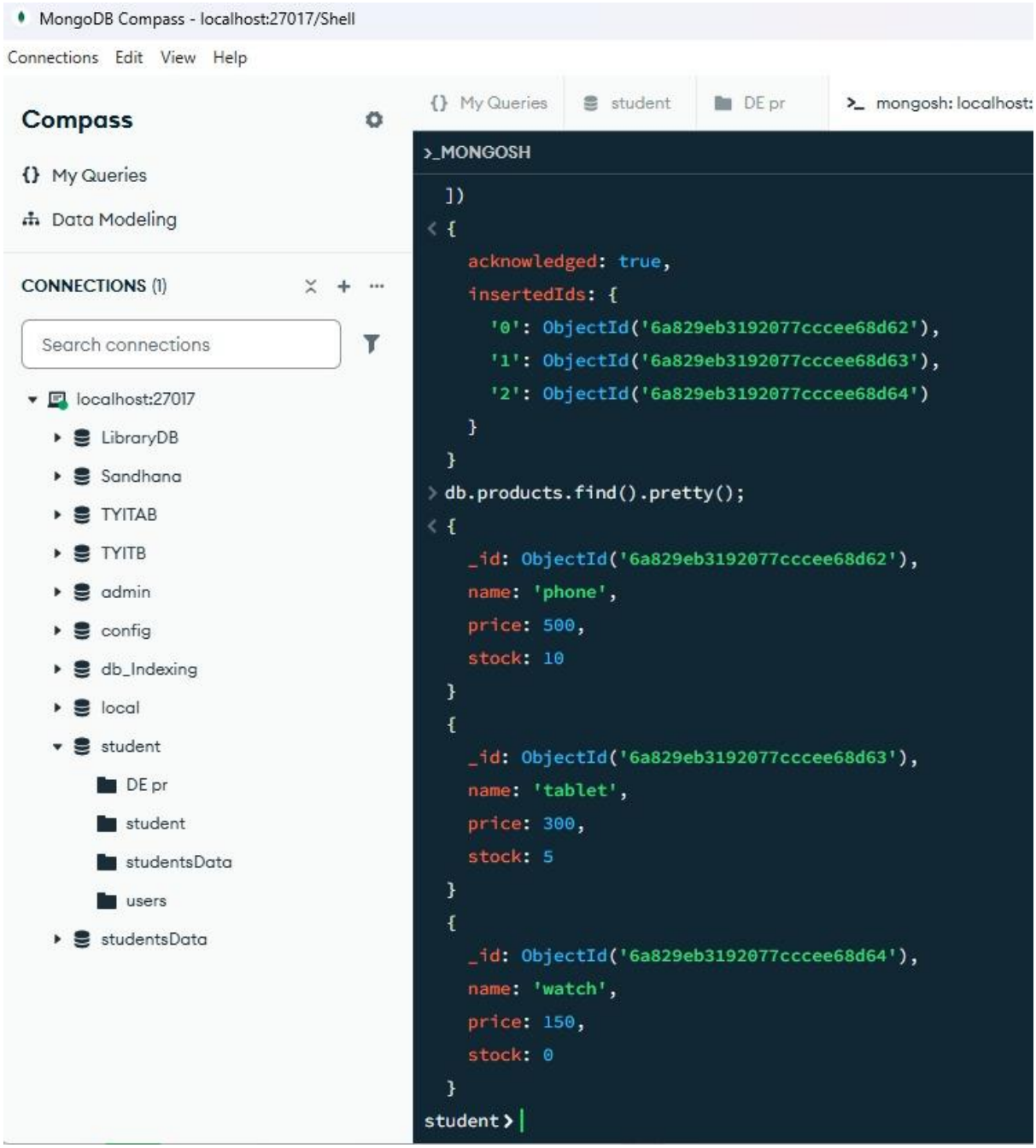

City-wise Sales:

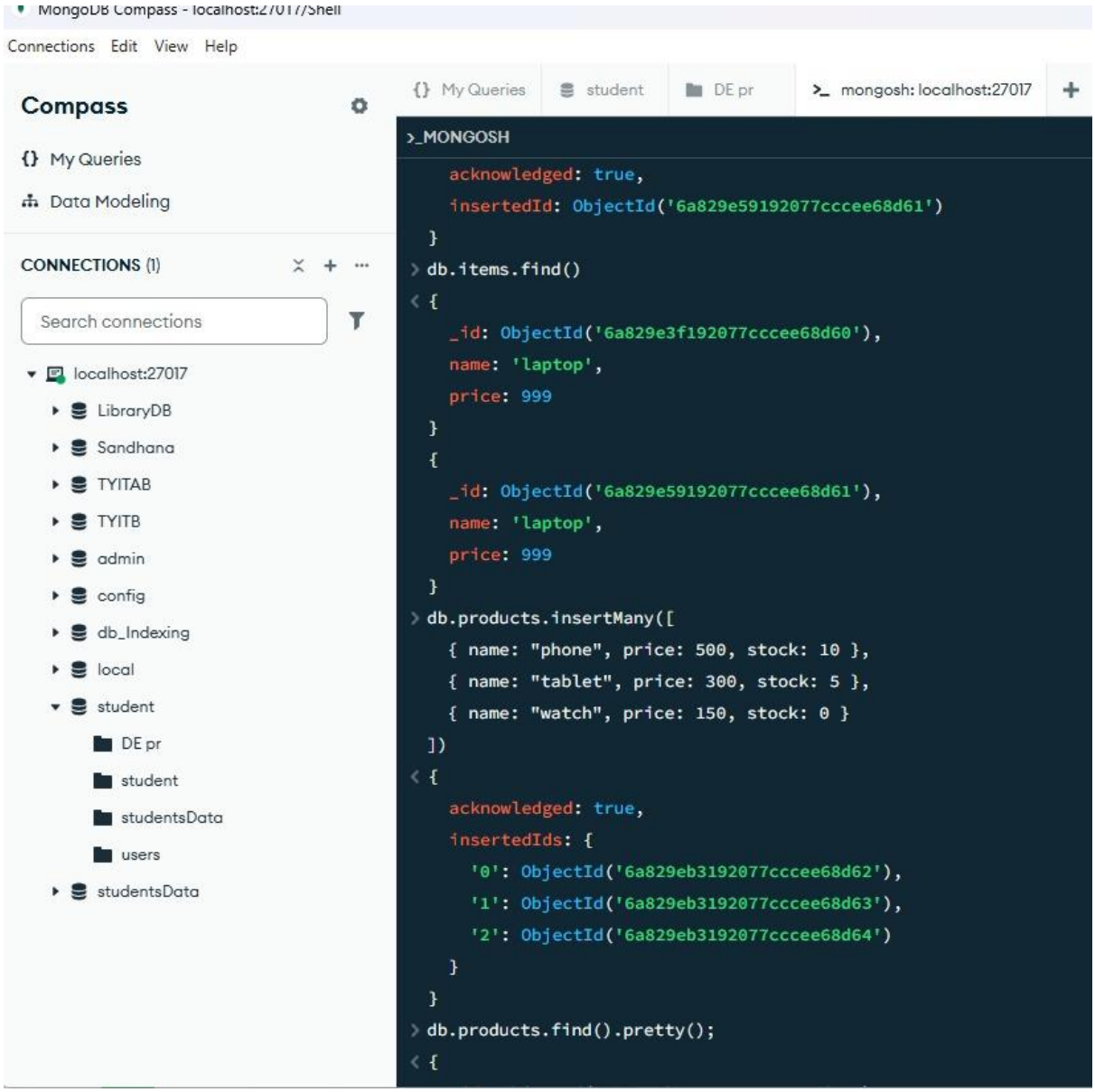
All SQL operations completed successfully and committed.
Database connection closed.
```

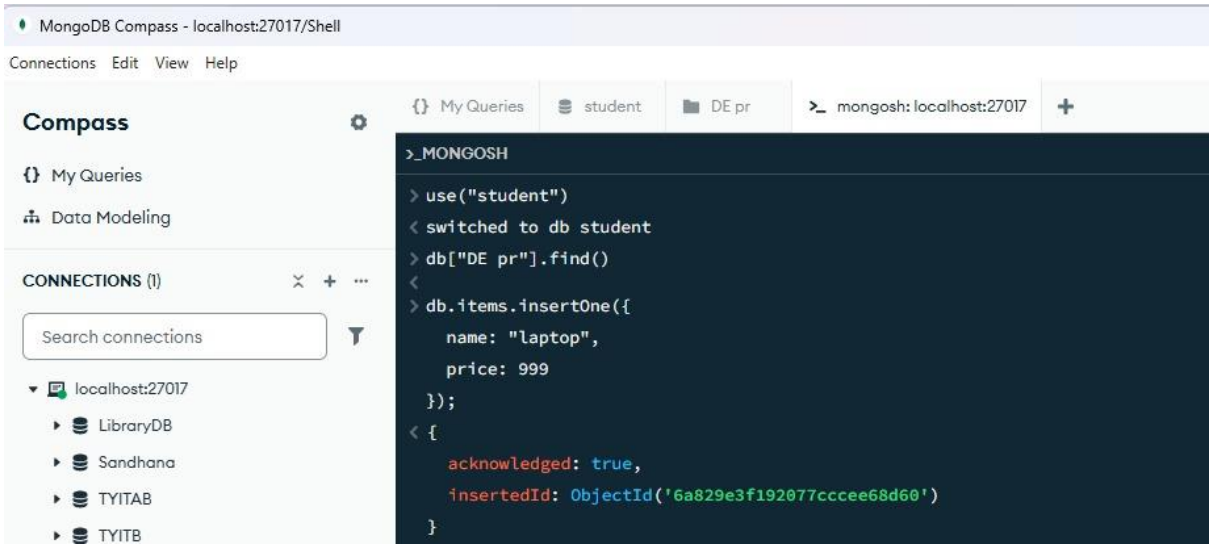
Result

Raw product data was staged, transformed into processed data, and analyzed for total sales, category-wise sales, and city-wise sales.

PRACTICAL 3


MongoDB Compass - localhost:27017/Shell
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
LibraryDB
Sandhana
TYITAB
TYITB
admin
config
db_Indexing
local
student
DE pr
student
studentsData
users
studentsData
My Queries
student
DE pr
mongosh: localhost:
>_MONGOSH
])
{
acknowledged: true,
insertedIds: {
'0': ObjectId('6a829eb3192077cccee68d62'),
'1': ObjectId('6a829eb3192077cccee68d63'),
'2': ObjectId('6a829eb3192077cccee68d64')
}
}
db.products.find().pretty();
{
_id: ObjectId('6a829eb3192077cccee68d62'),
name: 'phone',
price: 500,
stock: 10
}
{
_id: ObjectId('6a829eb3192077cccee68d63'),
name: 'tablet',
price: 300,
stock: 5
}
{
_id: ObjectId('6a829eb3192077cccee68d64'),
name: 'watch',
price: 150,
stock: 0
}
student>


MongoDB Compass - localhost:27017/Shell
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
LibraryDB
Sandhana
TYITAB
TYITB
admin
config
db_Indexing
local
student
DE pr
student
studentsData
users
studentsData
My Queries
student
DE pr
mongosh: localhost:27017
>_MONGOSH
acknowledged: true,
insertedId: ObjectId('6a829e59192077cccee68d61')
}
> db.items.find()
< {
_id: ObjectId('6a829e3f192077cccee68d60'),
name: 'laptop',
price: 999
}
{
_id: ObjectId('6a829e59192077cccee68d61'),
name: 'laptop',
price: 999
}
> db.products.insertMany([
{ name: "phone", price: 500, stock: 10 },
{ name: "tablet", price: 300, stock: 5 },
{ name: "watch", price: 150, stock: 0 }
])
< {
acknowledged: true,
insertedIds: {
'0': ObjectId('6a829eb3192077cccee68d62'),
'1': ObjectId('6a829eb3192077cccee68d63'),
'2': ObjectId('6a829eb3192077cccee68d64')
}
}
> db.products.find().pretty();
< {


MongoDB Compass - localhost:27017/Shell
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
LibraryDB
Sandhana
TYITAB
TYITB
My Queries
student
DE pr
mongosh: localhost:27017
>_MONGOSH
> use("student")
< switched to db student
> db["DE pr"].find()
<
> db.items.insertOne({
name: "laptop",
price: 999
});
< {
acknowledged: true,
insertedId: ObjectId('6a829e3f192077cccee68d60')
}

Practical : 4

QUE: Python program demonstrating Noise Elimination, Feature Selection, and Exploratory Data Analysis (EDA) using Pandas, NumPy, Matplotlib, and Scikit-learn

```
import pandas as pd
data = { 'Age': [20, 21, 22, 23, 24, 100],
   'Marks': [75, 80, 85, 90, 95, 20],
   'Attendance': [85, 88, 90, 92, 94, 50],
   'Constant': [1, 1, 1, 1, 1, 1]}
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
```

```
Original Dataset:
   Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1
5  100     20          50         1
```

```
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df_clean = df[(df['Age'] >= lower) & (df['Age'] <= upper)]
print("\nDataset after Noise Elimination:")
print(df_clean)
```

```
Dataset after Noise Elimination:
   Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1
```

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.0)

selected = selector.fit_transform(df_clean)

selected_columns = df_clean.columns[selector.get_support()]

print("Selected columns:")

print(selected_columns)
```

```
Selected columns:
Index(['Age', 'Marks', 'Attendance'], dtype='str')
```

```
print("\nDataset Information:")

print(df_clean.info())

print("\nStatistical Summary:")

print(df_clean.describe())

print("\nCorrelation Matrix:")

print(df_clean.corr())
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Age         5 non-null      int64
 1   Marks       5 non-null      int64
 2   Attendance  5 non-null      int64
 3   Constant    5 non-null      int64
dtypes: int64(4)
memory usage: 232.0 bytes
None

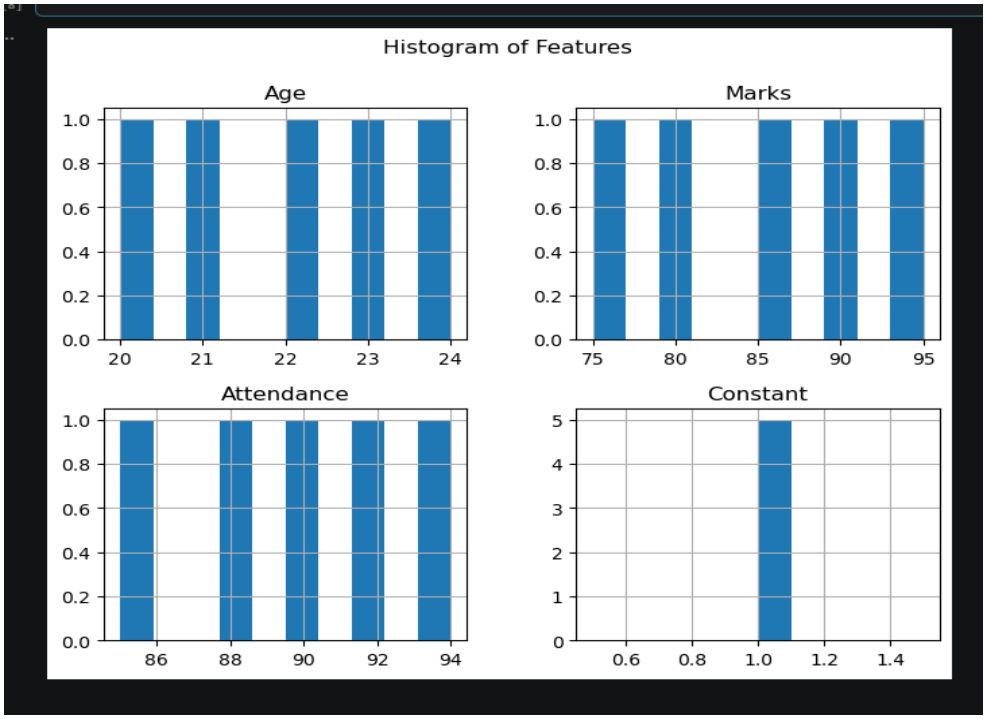
Statistical Summary:
             Age      Marks  Attendance  Constant
count   5.000000   5.000000     5.00000       5.0
mean   22.000000  85.000000    89.80000       1.0
std     1.581139   7.905694     3.49285       0.0
min    20.000000  75.000000    85.00000       1.0
25%    21.000000  80.000000    88.00000       1.0
50%    22.000000  85.000000    90.00000       1.0
75%    23.000000  90.000000    92.00000       1.0
max    24.000000  95.000000    94.00000       1.0
...
Age         1.000000  1.000000    0.995893       NaN
Marks       1.000000  1.000000    0.995893       NaN
Attendance  0.995893  0.995893    1.000000       NaN
Constant         NaN       NaN         NaN       NaN
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
import matplotlib.pyplot as plt

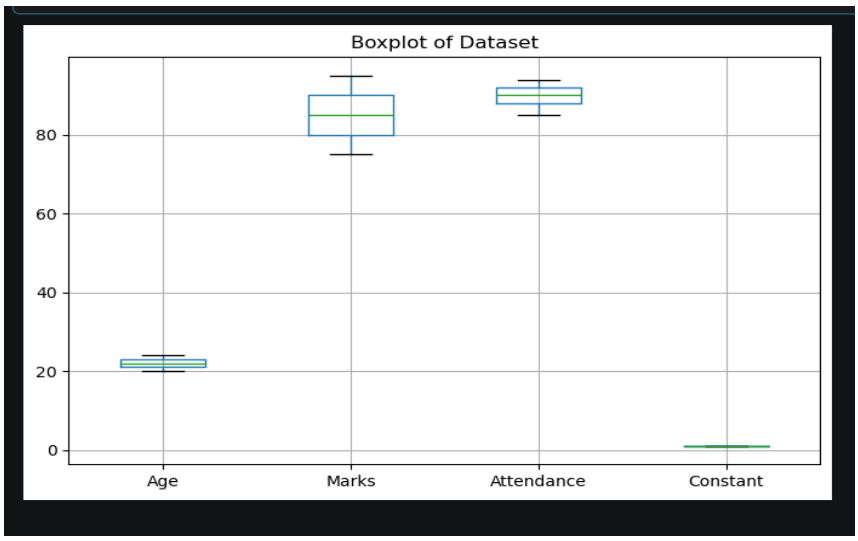
df_clean.hist(figsize=(8, 6))

plt.suptitle("Histogram of Features")

plt.show()
```




```
df_clean.boxplot(figsize=(8,5))
plt.title("Boxplot of Dataset")
plt.show()
```



Practical : 5

Que : Extracting Data from APIs and Flat Files Using Python1. ObjectiveWrite a Python script to fetch user data from a public REST API, clean it, and merge it with a flat CSV file containing location details.2. Code ImplementationCreate a script named data_extraction.py:pythonimport pandas as pd import requests

Code :

```
import pandas as pd
import requests
def extract_api_data(url):
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        return pd.json_normalize(data)
    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return pd.DataFrame()
def extract_csv_data(file_path):
    try:
        return pd.read_csv(file_path)
    except FileNotFoundError as e:
        print(f"File Error: {e}")
        return pd.DataFrame()
api_url = "https://jsonplaceholder.typicode.com/users"
api_df = extract_api_data(api_url)
```

```
print(api_df.head())
```

```
   id             name   username                    email  \
0   1     Leanne Graham       Bret          Sincere@april.biz
1   2      Ervin Howell  Antonette          Shanna@melissa.tv
2   3  Clementine Bauch   Samantha         Nathan@yesenia.net
3   4  Patricia Lebsack   Karianne  Julianne.OConner@kory.org
4   5  Chelsey Dietrich     Kamren   Lucio_Hettinger@annie.ca

                  phone        website     address.street address.suite  \
0  1-770-736-8031 x56442  hildegard.org        Kulas Light      Apt. 556
1    010-692-6593 x09125  anastasia.net      Victor Plains     Suite 879
2          1-463-123-4447    ramiro.info  Douglas Extension     Suite 847
3       493-170-9623 x156       kale.biz        Hoeger Mall      Apt. 692
4           (254)954-1289   demarco.info       Skiles Walks     Suite 351

    address.city address.zipcode address.geo.lat address.geo.lng  \
0   Gwenborough      92998-3874        -37.3159         81.1496
1   Wisokyburgh      90566-7771        -43.9509        -34.4618
2  McKenziehaven      59590-4157        -68.6102        -47.0653
3   South Elvis      53919-4257         29.4572       -164.2990
4    Roscoeview           33263        -31.8129         62.5342

         company.name                     company.catchPhrase  \
0    Romaguera-Crona   Multi-layered client-server neural-net
1       Deckow-Crist           Proactive didactic contingency
2  Romaguera-Jacobson        Face to face bifurcated interface
3      Robel-Corkery  Multi-tiered zero tolerance productivity
4        Keebler LLC      User-centric fault-tolerant solution

                          company.bs
0          harness real-time e-markets
1     synergize scalable supply-chains
2      e-enable strategic applications
3  transition cutting-edge web services
4     revolutionize end-to-end systems
```

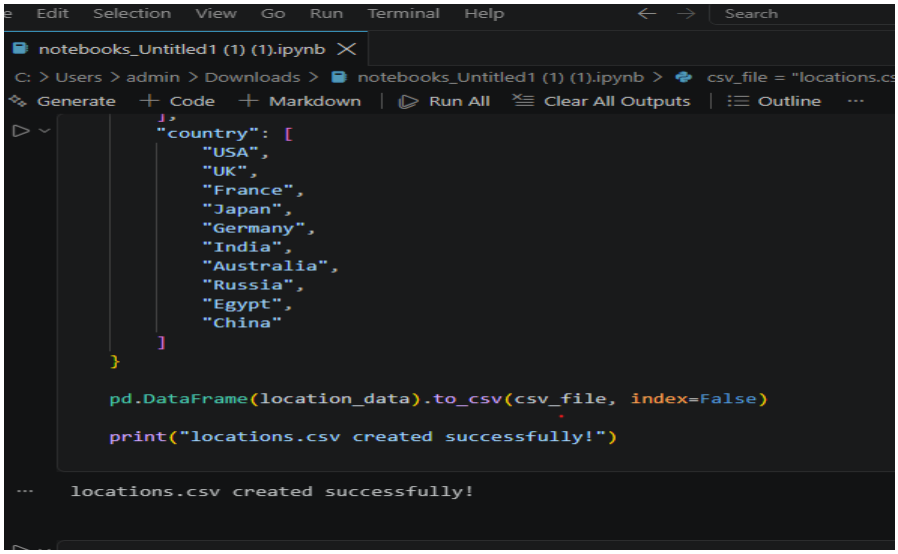
```
api_df = api_df[["id", "name", "email", "company.name"]]
api_df.rename(
    columns={"company.name": "company"},
    inplace=True)
print("Cleaned API Data:")
print(api_df)
```

```
print(api_df)
```

```
Cleaned API Data:
   id                     name                     email           company
0   1            Leanne Graham          Sincere@april.biz     Romaguera-Crona
1   2             Ervin Howell          Shanna@melissa.tv       Deckow-Crist
2   3         Clementine Bauch         Nathan@yesenia.net  Romaguera-Jacobson
3   4         Patricia Lebsack  Julianne.OConner@kory.org      Robel-Corkery
4   5         Chelsey Dietrich   Lucio_Hettinger@annie.ca        Keebler LLC
5   6      Mrs. Dennis Schulist    Karley_Dach@jasper.info  Considine-Lockman
6   7          Kurtis Weissnat     Telly.Hoeger@billy.biz        Johns Group
7   8  Nicholas Runolfsdottir V       Sherwood@rosamond.me    Abernathy Group
8   9            Glenna Reichert    Chaim_McDermott@dana.io      Yost and Sons
9  10         Clementina DuBuque     Rey.Padberg@karina.biz         Hoeger LLC
```

```
csv_file = "locations.csv"
location_data = {
    "id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "city": [
        "New York",
        "London",
        "Paris",
        "Tokyo",
        "Berlin",
        "Delhi",
        "Sydney",
```

```
        "Moscow",
        "Cairo",
        "Beijing" ],
    "country": [
        "USA",
        "UK",
        "France",
        "Japan",
        "Germany",
        "India",
        "Australia",
        "Russia",
        "Egypt",
        "China"]
}
pd.DataFrame(location_data).to_csv(csv_file, index=False)
print("locations.csv created successfully!")
```




```
csv_df = extract_csv_data(csv_file)
print("Location Data:")
print(csv_df)
```

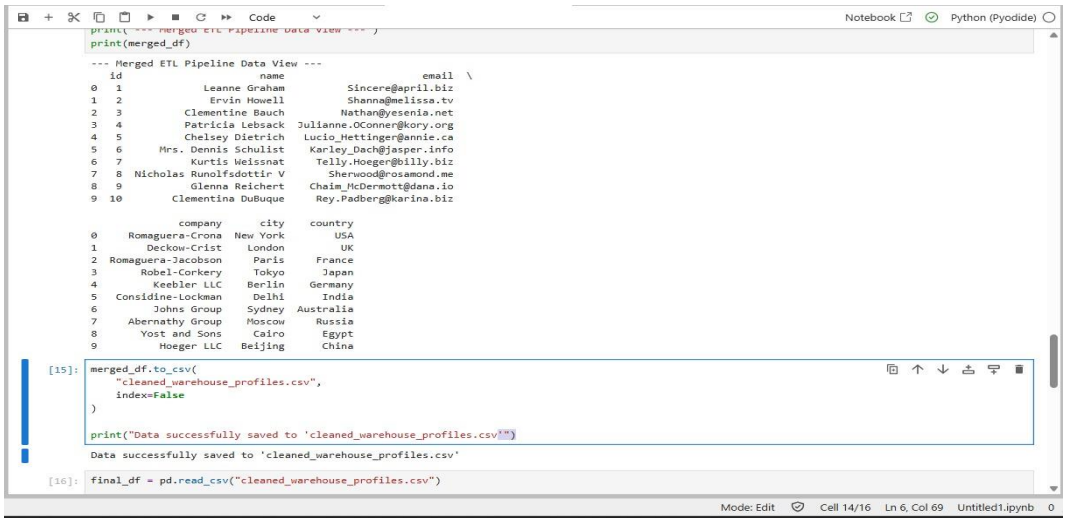
```
[13]: csv_df = extract_csv_data(csv_file)

      print("Location Data:")
      print(csv_df)
```

```
Location Data:
   id      city    country
0   1  New York        USA
1   2    London         UK
2   3     Paris     France
3   4     Tokyo      Japan
4   5    Berlin    Germany
5   6     Delhi      India
6   7    Sydney  Australia
7   8    Moscow     Russia
8   9     Cairo      Egypt
9  10   Beijing      China
```

```
[ ]:
```

```
merged_df = pd.merge(
    api_df,
    csv_df,
    on="id",
how="inner")
print("--- Merged ETL Pipeline Data View ---")
print(merged_df)
```



```
--- Merged ETL Pipeline Data View ---
   id                name                      email  \
0   1        Leanne Graham          Sincere@april.biz
1   2          Ervin Howell          Shanna@melissa.tv
2   3      Clementine Bauch         Nathan@yesenia.net
3   4      Patricia Lebsack  Julianne.OConner@kory.org
4   5      Chelsey Dietrich   Lucio_Hettinger@annie.ca
5   6   Mrs. Dennis Schulist    Karley_Dach@jasper.info
6   7        Kurtis Weissnat      Telly.Hoeger@billy.biz
7   8  Nicholas Runolfsdottir V     Sherwood@rosamond.me
8   9         Glenna Reichert    Chaim_McDermott@dana.io
9  10      Clementina DuBuque     Rey.Padberg@karina.biz

              company      city    country
0    Romaguera-Crona  New York        USA
1       Deckow-Crist    London         UK
2  Romaguera-Jacobson     Paris     France
3      Robel-Corkery     Tokyo      Japan
4        Keebler LLC    Berlin    Germany
5  Considine-Lockman     Delhi      India
6        Johns Group    Sydney  Australia
7    Abernathy Group    Moscow     Russia
8      Yost and Sons     Cairo      Egypt
9         Hoeger LLC   Beijing      China
```

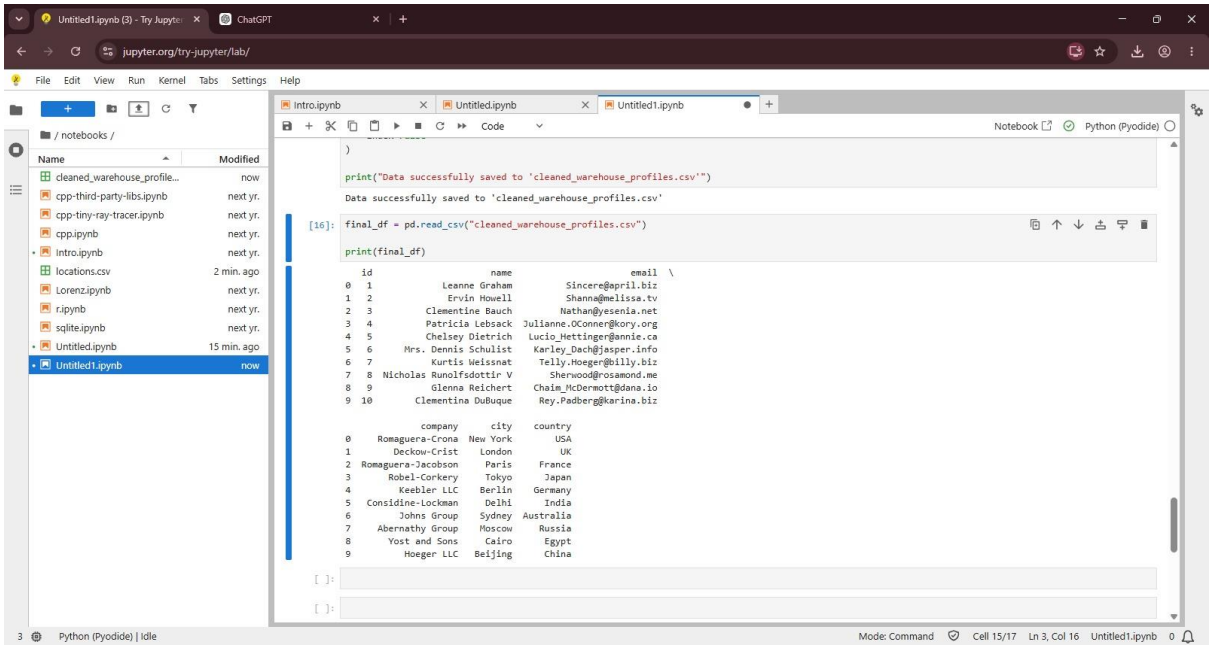
```
[15]: merged_df.to_csv(
          "cleaned_warehouse_profiles.csv",
          index=False
      )

      print("Data successfully saved to 'cleaned_warehouse_profiles.csv'")
```

```
Data successfully saved to 'cleaned_warehouse_profiles.csv'
```

```
[16]: final_df = pd.read_csv("cleaned_warehouse_profiles.csv")
```

```
final_df = pd.read_csv("cleaned_warehouse_profiles.csv")
print(final_df)
```


Untitled1.ipynb (3) - Try Jupyter
ChatGPT
jupyter.org/try-jupyter/lab/
File Edit View Run Kernel Tabs Settings Help
/ notebooks /
Name
Modified
cleaned_warehouse_profile... now
cpp-third-party-libs.ipynb next yr.
cpp-tiny-ray-tracer.ipynb next yr.
cpp.ipynb next yr.
intro.ipynb next yr.
locations.csv 2 min. ago
Lorenz.ipynb next yr.
r.ipynb next yr.
sqlite.ipynb next yr.
Untitled.ipynb 15 min. ago
Untitled1.ipynb now
intro.ipynb
Untitled.ipynb
Untitled1.ipynb
Code
Notebook
Python (Pyodide)
print("Data successfully saved to 'cleaned_warehouse_profiles.csv'")
Data successfully saved to 'cleaned_warehouse_profiles.csv'
[16]:
final_df = pd.read_csv("cleaned_warehouse_profiles.csv")
print(final_df)
id name email \
0 1 Leanne Graham Sincere@april.biz
1 2 Ervin Howell Shanna@melissa.tv
2 3 Clementine Bauch Nathan@yesenia.net
3 4 Patricia Lebsack Julianne.OConner@kory.org
4 5 Chelsey Dietrich Lucio_Hettinger@annie.ca
5 6 Mrs. Dennis Schulist Karley_Dach@jasper.info
6 7 Kurtis Weissnat Telly.Hoeger@billy.biz
7 8 Nicholas Runolfsdottir V Sherwood@rosamond.me
8 9 Glenna Reichert Chaim_McDermott@dana.io
9 10 Clementina DuBuque Rey.Padberg@karina.biz
company city country
0 Romaguera-Crona New York USA
1 Deckow-Crist London UK
2 Romaguera-Jacobson Paris France
3 Robel-Corkery Tokyo Japan
4 Keebler LLC Berlin Germany
5 Considine-Lockman Delhi India
6 Johns Group Sydney Australia
7 Abernathy Group Moscow Russia
8 Yost and Sons Cairo Egypt
9 Hoeger LLC Beijing China
[ ]:
[ ]:
3 Python (Pyodide) | Idle
Mode: Command
Cell 15/17 Ln 3, Col 16 Untitled1.ipynb 0

Practical 6

Aim

To configure an Apache Airflow Directed Acyclic Graph (DAG) that runs daily and executes sequential tasks for starting the pipeline, running a data-extraction script, and logging successful completion.

Code

```
#Que: Setting up Apache Airflow and Creating a DAG1. ObjectiveConfigure a Directed Acyclic Graph (DAG) in Apache Airflow to run daily, executing sequential dependency tasks: a start notifier, an extraction script, and a pipeline success logger.2. DAG ImplementationPlace this code in your Airflow DAGs directory (usually ~/airflow/dags/data_pipeline_dag.py):pythonfrom datetime import datetime, timedelta
from datetime import datetime, timedelta

# Install required packages, including explicit installation of rfc3987
!pip install --upgrade pip
!pip install rfc3987 apache-airflow --upgrade

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator


# Default arguments
default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}


# Define DAG
with DAG(
    dag_id="university_etl_orchestration",
    default_args=default_args,
    description="Daily ETL pipeline orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:

    # Task 1: Start pipeline
    start_pipeline = EmptyOperator(
        task_id="start_pipeline"
    )

    # Task 2: Run extraction script
    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="python3 /home/$USER/airflow/dags/data_extraction.py",
    )

    # Task 3: Log successful completion
    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully at $(date)"',
    )

    # Task dependencies
    start_pipeline >> execute_extraction >> pipeline_complete
```

Output

```
Successfully installed pip-26.2.1

2026-10-01T08:49:45.803200Z [warning  ] The `airflow.operators.bash.BashOperator` attribute is deprecated. Please use `'airflow.providers.standard.operators.bash.BashOperator'`. [py.warnings] category=DeprecatedImportWarning filename=/tmp/ipykernel_2824/3822651003.py lineno=9
2026-10-01T08:49:45.810016Z [warning  ] The `airflow.operators.empty.EmptyOperator` attribute is deprecated. Please use
```

```
``'airflow.providers.standard.operators.empty.EmptyOperator'`. [py.warnings] category=DeprecatedImportWarning
filename=/tmp/ipykernel_2824/3822651003.py lineno=10
```

```
from datetime import datetime

print("Starting data extraction...")

data = [
    {"id": 1, "name": "Student A"},
    {"id": 2, "name": "Student B"},
    {"id": 3, "name": "Student C"}
]

print("Extracted data:")

for record in data:
    print(record)

print("Extraction completed successfully.")
print("Execution time:", datetime.now())
```

Output

```
Starting data extraction...
Extracted data:
{'id': 1, 'name': 'Student A'}
{'id': 2, 'name': 'Student B'}
{'id': 3, 'name': 'Student C'}
Extraction completed successfully.
Execution time: 2026-10-01 08:51:26.983305
```

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator


default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}


with DAG(
    dag_id="university_etl_orchestration",
    default_args=default_args,
    description="Daily ETL pipeline orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:

    start_pipeline = EmptyOperator(
        task_id="start_pipeline"
    )

    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="python3 /home/$USER/airflow/dags/data_extraction.py",
    )

    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully at $(date)"',
    )

    start_pipeline >> execute_extraction >> pipeline_complete
```

Output

```
2026-10-01T08:51:47.868897Z [warning  ] The `airflow.operators.bash.BashOperator` attribute is deprecated. Please use
``'airflow.providers.standard.operators.bash.BashOperator'`. [py.warnings] category=DeprecatedImportWarning
```

```
filename=/tmp/ipykernel_3460/2383551127.py lineno=4
2026-10-01T08:51:47.875607Z [warning  ] The `airflow.operators.empty.EmptyOperator` attribute is deprecated. Please use
``airflow.providers.standard.operators.empty.EmptyOperator''. [py.warnings] category=DeprecatedImportWarning
filename=/tmp/ipykernel_3460/2383551127.py lineno=5
```

```
from datetime import datetime

print("====================================")
print("       DATA EXTRACTION STARTED")
print("====================================")

# Sample data
data = [
    {"id": 1, "name": "Student A"},
    {"id": 2, "name": "Student B"},
    {"id": 3, "name": "Student C"}
]

print("Extracting data...")

for record in data:
    print(record)

print("Data extraction completed successfully.")

print("Execution time:", datetime.now())

print("====================================")
print("       DATA EXTRACTION COMPLETED")
print("====================================")
```

Output

```
====================================
       DATA EXTRACTION STARTED
====================================
Extracting data...
{'id': 1, 'name': 'Student A'}
{'id': 2, 'name': 'Student B'}
{'id': 3, 'name': 'Student C'}
Data extraction completed successfully.
Execution time: 2026-10-01 08:51:56.002598
====================================
       DATA EXTRACTION COMPLETED
====================================
```

Result

The DAG defines the sequential dependency: start_pipeline → run_extraction_script → log_pipeline_success.

Practical 7

Aim

To implement ETL pipelines using CSV and JSON data, including data cleaning, transformation, validation, loading into SQLite databases, removal of invalid records, and incremental data loading.

Code

7.1 ETL Pipeline from CSV to Database

```
# ETL Pipeline from CSV to Database
import pandas as pd
import sqlite3

# ----------------------------
# EXTRACT
# ----------------------------
data = {
    "Name": ["Amit", "Priya", "Rahul", "Sneha", "Karan"],
    "Age": [21, 22, None, 23, 25],
    "City": ["Mumbai", "Pune", "Mumbai", "Delhi", "Pune"],
    "Salary": [30000, 35000, 40000, None, 45000]
}

df = pd.DataFrame(data)
df.to_csv("employees.csv", index=False)

# Read CSV file
df = pd.read_csv("employees.csv")

print("Extracted Data:")
print(df)

# ----------------------------
# TRANSFORM / CLEAN
# ----------------------------

# Fill missing Age with mean
df["Age"] = df["Age"].fillna(df["Age"].mean())

# Fill missing Salary with median
df["Salary"] = df["Salary"].fillna(df["Salary"].median())

# Convert Age to integer
df["Age"] = df["Age"].astype(int)

# Remove duplicate records
df = df.drop_duplicates()

# Create Salary Category
df["Salary_Category"] = df["Salary"].apply(
    lambda x: "High" if x >= 40000 else "Low"
)

print("\nTransformed Data:")
print(df)

# ----------------------------
# LOAD
# ----------------------------

conn = sqlite3.connect("etl_database.db")

df.to_sql(
    "employees",
    conn,
    if_exists="replace",
    index=False
)

print("\nData successfully loaded into database.")
```

```
# Verify
result = pd.read_sql("SELECT * FROM employees", conn)
print("\nDatabase Data:")
print(result)

conn.close()
```

Output

```
Extracted Data:
    Name   Age    City   Salary
0   Amit  21.0  Mumbai  30000.0
1  Priya  22.0    Pune  35000.0
2  Rahul   NaN  Mumbai  40000.0
3  Sneha  23.0   Delhi      NaN
4  Karan  25.0    Pune  45000.0

Transformed Data:
    Name  Age    City   Salary Salary_Category
0   Amit   21  Mumbai  30000.0             Low
1  Priya   22    Pune  35000.0             Low
2  Rahul   22  Mumbai  40000.0            High
3  Sneha   23   Delhi  37500.0             Low
4  Karan   25    Pune  45000.0            High

Data successfully loaded into database.

Database Data:
    Name  Age    City   Salary Salary_Category
0   Amit   21  Mumbai  30000.0             Low
1  Priya   22    Pune  35000.0             Low
2  Rahul   22  Mumbai  40000.0            High
3  Sneha   23   Delhi  37500.0             Low
4  Karan   25    Pune  45000.0            High
```

7.2 Extract Multiple CSV Files and Combine Them

```
# Extract Multiple CSV Files and Combine Them
import pandas as pd
import sqlite3

# ------------------------------
# CREATE SAMPLE CSV FILES
# ------------------------------

data1 = {
    "ID": [1, 2, 3],
    "Name": ["Amit", "Priya", "Rahul"],
    "Marks": [85, 90, 78]
}

data2 = {
    "ID": [4, 5, 6],
    "Name": ["Sneha", "Karan", "Neha"],
    "Marks": [88, 76, 95]
}

pd.DataFrame(data1).to_csv("students1.csv", index=False)
pd.DataFrame(data2).to_csv("students2.csv", index=False)

# ------------------------------
# EXTRACT
# ------------------------------

df1 = pd.read_csv("students1.csv")
df2 = pd.read_csv("students2.csv")

print("File 1:")
print(df1)

print("\nFile 2:")
print(df2)

# ------------------------------
# TRANSFORM
# ------------------------------
```

```
combined_df = pd.concat([df1, df2], ignore_index=True)

# Create result column
combined_df["Result"] = combined_df["Marks"].apply(
    lambda x: "Pass" if x >= 40 else "Fail"
)

print("\nCombined Dataset:")
print(combined_df)

# ----------------------------
# LOAD
# ----------------------------

conn = sqlite3.connect("student_database.db")

combined_df.to_sql(
    "students",
    conn,
    if_exists="replace",
    index=False
)

print("\nCombined data loaded successfully.")

conn.close()
```

Output

```
File 1:
   ID   Name  Marks
0   1   Amit     85
1   2  Priya     90
2   3  Rahul     78

File 2:
   ID   Name  Marks
0   4  Sneha     88
1   5  Karan     76
2   6   Neha     95

Combined Dataset:
   ID   Name  Marks Result
0   1   Amit     85   Pass
1   2  Priya     90   Pass
2   3  Rahul     78   Pass
3   4  Sneha     88   Pass
4   5  Karan     76   Pass
5   6   Neha     95   Pass

Combined data loaded successfully.
```

7.3 ETL Pipeline for JSON Data

```
#ETL Pipeline for JSON Data
import json
import pandas as pd
import sqlite3

# ----------------------------
# CREATE SAMPLE JSON DATA
# ----------------------------

json_data = [
    {
        "id": 1,
        "name": "Amit",
        "email": "amit@gmail.com",
        "address": {
            "city": "Mumbai",
            "country": "India"
        }
    },
    {
        "id": 2,
        "name": "Priya",
        "email": "priya@gmail.com",
```

```
        "address": {
            "city": "Pune",
            "country": "India"
        }
    }
]

with open("users.json", "w") as file:
    json.dump(json_data, file, indent=4)

# ------------------------------
# EXTRACT
# ------------------------------

with open("users.json", "r") as file:
    data = json.load(file)

print("Extracted JSON:")
print(data)

# ------------------------------
# TRANSFORM
# ------------------------------

transformed_data = []

for user in data:
    transformed_data.append({
        "User_ID": user["id"],
        "Name": user["name"].upper(),
        "Email": user["email"].lower(),
        "City": user["address"]["city"],
        "Country": user["address"]["country"]
    })

df = pd.DataFrame(transformed_data)

print("\nTransformed Data:")
print(df)

# ------------------------------
# LOAD
# ------------------------------

conn = sqlite3.connect("json_database.db")

df.to_sql(
    "users",
    conn,
    if_exists="replace",
    index=False
)

print("\nJSON data loaded into relational database.")

# Verify
result = pd.read_sql("SELECT * FROM users", conn)
print("\nDatabase Table:")
print(result)

conn.close()
```

Output

```
Extracted JSON:
[{'id': 1, 'name': 'Amit', 'email': 'amit@gmail.com', 'address': {'city': 'Mumbai', 'country': 'India'}}, {'id': 2, 'name':
'Priya', 'email': 'priya@gmail.com', 'address': {'city': 'Pune', 'country': 'India'}}]

Transformed Data:
   User_ID   Name            Email    City Country
0        1   AMIT   amit@gmail.com  Mumbai   India
1        2  PRIYA  priya@gmail.com    Pune   India

JSON data loaded into relational database.

Database Table:
```

```
   User_ID   Name            Email    City Country
0        1   AMIT   amit@gmail.com  Mumbai   India
1        2  PRIYA  priya@gmail.com    Pune   India
```

7.4 ETL Process to Identify and Remove Invalid Records

```
#ETL Process to Identify and Remove Invalid Records
import pandas as pd
import sqlite3

# ----------------------------
# EXTRACT
# ----------------------------

data = {
    "ID": [1, 2, 3, 4, 5],
    "Name": ["Amit", "Priya", "", "Rahul", "Sneha"],
    "Age": [21, -5, 25, 200, 23],
    "Email": [
        "amit@gmail.com",
        "priya@gmail.com",
        "invalidemail",
        "rahul@gmail.com",
        "sneha@gmail.com"
    ]
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)

# ----------------------------
# IDENTIFY INVALID RECORDS
# ----------------------------

invalid = (
    (df["Name"] == "") |
    (df["Age"] < 0) |
    (df["Age"] > 120) |
    (~df["Email"].str.contains("@", na=False))
)

invalid_records = df[invalid]

print("\nInvalid Records:")
print(invalid_records)

# ----------------------------
# REMOVE INVALID RECORDS
# ----------------------------

clean_df = df[~invalid].copy()

print("\nClean Data:")
print(clean_df)

# ----------------------------
# LOAD
# ----------------------------

conn = sqlite3.connect("valid_data.db")

clean_df.to_sql(
    "valid_users",
    conn,
    if_exists="replace",
    index=False
)

print("\nValid records loaded successfully.")

conn.close()
```

Output

```
Original Data:
   ID   Name  Age            Email
0   1   Amit   21   amit@gmail.com
1   2  Priya   -5  priya@gmail.com
2   3           25     invalidemail
3   4  Rahul  200  rahul@gmail.com
4   5  Sneha   23  sneha@gmail.com

Invalid Records:
   ID   Name  Age            Email
1   2  Priya   -5  priya@gmail.com
2   3           25     invalidemail
3   4  Rahul  200  rahul@gmail.com

Clean Data:
   ID   Name  Age            Email
0   1   Amit   21   amit@gmail.com
4   5  Sneha   23  sneha@gmail.com

Valid records loaded successfully.
```

7.5 Data Validation Before Loading

```
#Data Validation Before Loading
import pandas as pd
import sqlite3

# ------------------------------
# EXTRACT
# ------------------------------

data = {
    "ID": [1, 2, 3, 4],
    "Name": ["Amit", "Priya", "Rahul", "Sneha"],
    "Age": [21, 22, 150, 24],
    "Email": [
        "amit@gmail.com",
        "priya@gmail.com",
        "rahul@gmail.com",
        "invalid-email"
    ]
}

df = pd.DataFrame(data)

print("Extracted Data:")
print(df)

# ------------------------------
# VALIDATION
# ------------------------------

def validate_record(row):

    if row["Age"] < 18 or row["Age"] > 100:
        return False

    if "@" not in row["Email"]:
        return False

    if row["Name"] == "":
        return False

    return True


df["Valid"] = df.apply(validate_record, axis=1)

print("\nValidation Results:")
print(df)

# Separate valid and invalid records
valid_df = df[df["Valid"] == True].drop(columns=["Valid"])
invalid_df = df[df["Valid"] == False].drop(columns=["Valid"])

print("\nValid Records:")
```

```
print(valid_df)

print("\nInvalid Records:")
print(invalid_df)

# ------------------------------
# LOAD ONLY VALID DATA
# ------------------------------

conn = sqlite3.connect("validated_database.db")

valid_df.to_sql(
    "customers",
    conn,
    if_exists="replace",
    index=False
)

print("\nValidated data loaded successfully.")

conn.close()
```

Output

```
Extracted Data:
   ID   Name  Age            Email
0   1   Amit   21   amit@gmail.com
1   2  Priya   22  priya@gmail.com
2   3  Rahul  150  rahul@gmail.com
3   4  Sneha   24    invalid-email

Validation Results:
   ID   Name  Age            Email  Valid
0   1   Amit   21   amit@gmail.com   True
1   2  Priya   22  priya@gmail.com   True
2   3  Rahul  150  rahul@gmail.com  False
3   4  Sneha   24    invalid-email  False

Valid Records:
   ID   Name  Age            Email
0   1   Amit   21   amit@gmail.com
1   2  Priya   22  priya@gmail.com

Invalid Records:
   ID   Name  Age            Email
2   3  Rahul  150  rahul@gmail.com
3   4  Sneha   24    invalid-email

Validated data loaded successfully.
```

7.6 Incremental Data Loading

```
#Incremental Data Loading
import pandas as pd
import sqlite3

# ------------------------------
# INITIAL DATA
# ------------------------------

initial_data = {
    "ID": [1, 2, 3],
    "Name": ["Amit", "Priya", "Rahul"],
    "Salary": [30000, 35000, 40000]
}

df_initial = pd.DataFrame(initial_data)

df_initial.to_csv("employees_initial.csv", index=False)

# ------------------------------
# CREATE DATABASE
# ------------------------------

conn = sqlite3.connect("incremental_database.db")

df_initial.to_sql(
```

```
    "employees",
    conn,
    if_exists="replace",
    index=False
)

print("Initial data loaded:")
print(df_initial)

# ----------------------------
# NEW DATA
# ----------------------------

new_data = {
    "ID": [4, 5],
    "Name": ["Sneha", "Karan"],
    "Salary": [42000, 45000]
}

df_new = pd.DataFrame(new_data)

df_new.to_csv("employees_new.csv", index=False)

print("\nNew data:")
print(df_new)

# ----------------------------
# CHECK EXISTING RECORDS
# ----------------------------

existing_df = pd.read_sql(
    "SELECT ID FROM employees",
    conn
)

print("\nExisting IDs:")
print(existing_df)

# ----------------------------
# INCREMENTAL LOAD
# ----------------------------

incremental_df = df_new[
    ~df_new["ID"].isin(existing_df["ID"])
]

if not incremental_df.empty:

    incremental_df.to_sql(
        "employees",
        conn,
        if_exists="append",
        index=False
    )

    print("\nNew records inserted:")
    print(incremental_df)

else:
    print("\nNo new records found.")

# ----------------------------
# FINAL DATABASE
# ----------------------------

final_df = pd.read_sql(
    "SELECT * FROM employees",
    conn
)

print("\nFinal Database:")
print(final_df)

conn.close()
```

Output

```
Initial data loaded:
   ID   Name  Salary
0   1   Amit   30000
1   2  Priya   35000
2   3  Rahul   40000

New data:
   ID   Name  Salary
0   4  Sneha   42000
1   5  Karan   45000

Existing IDs:
   ID
0   1
1   2
2   3

New records inserted:
   ID   Name  Salary
0   4  Sneha   42000
1   5  Karan   45000

Final Database:
   ID   Name  Salary
0   1   Amit   30000
1   2  Priya   35000
2   3  Rahul   40000
3   4  Sneha   42000
4   5  Karan   45000
```

Result

The notebook demonstrates CSV/JSON extraction, transformation, validation and cleaning, SQLite loading, and incremental insertion of new records.

PRACTICAL 8

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.4)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, sum, count
spark = SparkSession.builder \
    .appName("PySpark Assignment") \
    .getOrCreate()

print("Spark started successfully!")
```

```
Spark started successfully!
```

```
%%writefile sales.csv
Product,Category,Sales,Quantity
Laptop,Electronics,1000,2
Phone,Electronics,500,5
Chair,Furniture,200,4
Table,Furniture,300,2
Laptop,Electronics,1000,2
Headphones,Electronics,150,10
Chair,Furniture,200,4
Book,Stationery,50,20
```

```
Writing sales.csv
```

```
df = spark.read.csv("sales.csv", header=True, inferSchema=True)

df.show()

print("Schema:")
df.printSchema()
```

```
+----------+-----------+-----+--------+
|   Product|   Category|Sales|Quantity|
+----------+-----------+-----+--------+
|    Laptop|Electronics| 1000|       2|
|     Phone|Electronics|  500|       5|
|     Chair|  Furniture|  200|       4|
|     Table|  Furniture|  300|       2|
|    Laptop|Electronics| 1000|       2|
|Headphones|Electronics|  150|      10|
|     Chair|  Furniture|  200|       4|
|      Book| Stationery|   50|      20|
+----------+-----------+-----+--------+

Schema:
root
 |-- Product: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sales: integer (nullable = true)
 |-- Quantity: integer (nullable = true)
```

```
filtered_df = df.filter(col("Sales") > 500)

filtered_df.show()
```

```
+-------+-----------+-----+--------+
|Product|   Category|Sales|Quantity|
+-------+-----------+-----+--------+
| Laptop|Electronics| 1000|       2|
| Laptop|Electronics| 1000|       2|
+-------+-----------+-----+--------+
```

```
df.filter(df.Sales > 500).show()
```

```
+-------+-----------+-----+--------+
|Product|   Category|Sales|Quantity|
+-------+-----------+-----+--------+
| Laptop|Electronics| 1000|       2|
| Laptop|Electronics| 1000|       2|
+-------+-----------+-----+--------+
```

```
grouped_df = df.groupBy("Category").agg(
    sum("Sales").alias("Total_Sales")
)

grouped_df.show()
```

```
+-----------+-----------+
|   Category|Total_Sales|
+-----------+-----------+
| Stationery|         50|
|Electronics|       2650|
|  Furniture|        700|
+-----------+-----------+
```

```
grouped_df = df.groupBy("Category").sum("Sales")
```

```
+-----------+----------+
|   Category|sum(Sales)|
+-----------+----------+
| Stationery|        50|
|Electronics|      2650|
|  Furniture|       700|
+-----------+----------+
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, sum

spark = SparkSession.builder \
    .appName("Sales Analysis") \
    .getOrCreate()

df = spark.read.csv(
    "sales.csv",
    header=True,
    inferSchema=True
)

df.show()
```

```
+----------+-----------+-----+--------+
|   Product|   Category|Sales|Quantity|
+----------+-----------+-----+--------+
|    Laptop|Electronics| 1000|       2|
|     Phone|Electronics|  500|       5|
|     Chair|  Furniture|  200|       4|
|     Table|  Furniture|  300|       2|
|    Laptop|Electronics| 1000|       2|
|Headphones|Electronics|  150|      10|
|     Chair|  Furniture|  200|       4|
|      Book| Stationery|   50|      20|
+----------+-----------+-----+--------+
```

```
from pyspark.sql.functions import sum

df.groupBy("Category").agg(
    sum("Sales").alias("Total_Sales")
).show()
```

```
+-----------+-----------+
|   Category|Total_Sales|
+-----------+-----------+
| Stationery|         50|
|Electronics|       2650|
|  Furniture|        700|
+-----------+-----------+
```

```
df.dropDuplicates().show()
```

```
+----------+-----------+-----+--------+
|   Product|   Category|Sales|Quantity|
+----------+-----------+-----+--------+
|      Book| Stationery|   50|      20|
|    Laptop|Electronics| 1000|       2|
|     Table|  Furniture|  300|       2|
|     Phone|Electronics|  500|       5|
|     Chair|  Furniture|  200|       4|
|Headphones|Electronics|  150|      10|
+----------+-----------+-----+--------+
```

```
from pyspark.sql.functions import sum, avg
```

```
grouped_df = df.groupBy("Category").agg(
    sum("Sales").alias("Total_Sales")
)

print("Total Sales by Category:")
grouped_df.show()
```

```
Total Sales by Category:
+-----------+-----------+
|   Category|Total_Sales|
+-----------+-----------+
| Stationery|         50|
|Electronics|       2650|
|  Furniture|        700|
+-----------+-----------+
```

```
deduplicated_df = df.dropDuplicates()

print("Dataset after removing duplicates:")
deduplicated_df.show()
```

```
Dataset after removing duplicates:
+----------+-----------+-----+--------+
|   Product|   Category|Sales|Quantity|
+----------+-----------+-----+--------+
|      Book| Stationery|   50|      20|
|    Laptop|Electronics| 1000|       2|
|     Table|  Furniture|  300|       2|
|     Phone|Electronics|  500|       5|
|     Chair|  Furniture|  200|       4|
|Headphones|Electronics|  150|      10|
+----------+-----------+-----+--------+
```

```
%%writefile product_info.csv
Product,Brand
Laptop,Dell
Phone,Samsung
Chair,IKEA
Table,IKEA
Headphones,Sony
Book,Penguin
```

```
Writing product_info.csv
```

```
product_df = spark.read.csv(
    "product_info.csv",
    header=True,
    inferSchema=True
)

product_df.show()
```

```
+----------+-------+
|   Product|  Brand|
+----------+-------+
|    Laptop|   Dell|
|     Phone|Samsung|
|     Chair|   IKEA|
|     Table|   IKEA|
|Headphones|   Sony|
|      Book|Penguin|
+----------+-------+
```

```
joined_df = df.join(
    product_df,
    on="Product",
    how="inner"
)

print("Joined Dataset:")
joined_df.show()
```

```
Joined Dataset:
+----------+-----------+-----+--------+-------+
|   Product|   Category|Sales|Quantity|  Brand|
+----------+-----------+-----+--------+-------+
|    Laptop|Electronics| 1000|       2|   Dell|
|     Phone|Electronics|  500|       5|Samsung|
|     Chair|  Furniture|  200|       4|   IKEA|
|     Table|  Furniture|  300|       2|   IKEA|
|    Laptop|Electronics| 1000|       2|   Dell|
|Headphones|Electronics|  150|      10|   Sony|
|     Chair|  Furniture|  200|       4|   IKEA|
|      Book| Stationery|   50|      20|Penguin|
+----------+-----------+-----+--------+-------+
```

```
average_sales = df.groupBy("Category").agg(
    avg("Sales").alias("Average_Sales")
)

print("Average Sales by Product Category:")
average_sales.show()
```

```
Average Sales by Product Category:
+-----------+------------------+
|   Category|     Average_Sales|
+-----------+------------------+
| Stationery|              50.0|
|Electronics|             662.5|
|  Furniture|233.33333333333334|
+-----------+------------------+
```

Practical-9

Que: Build an end-to-end pipeline: CSV → Python/Pandas → Data Cleaning → SQL Database → Report.
Build an ETL pipeline for an e-commerce dataset containing customers, orders, products, and payments.
Design a data pipeline that handles both historical and newly arriving data.

1. **CSV → Pandas → Data Cleaning → SQL Database → Report**

```
import pandas as pd
data = { "Customer": ["John", "Mary", "John", "David", None],
    "Product": ["Laptop", "Phone", "Laptop", "Tablet", "Phone"],
    "Sales": [1000, 500, 1000, 700, None],
    "Quantity": [1, 2, 1, 3, 2]}
df = pd.DataFrame(data)
df.to_csv("sales_data.csv", index=False)
print("CSV created successfully!")
df
```

```
CSV created successfully!
```

	Customer	Product	Sales	Quantity
0	John	Laptop	1000.0	1
1	Mary	Phone	500.0	2
2	John	Laptop	1000.0	1
3	David	Tablet	700.0	3
4	None	Phone	NaN	2

```
df = pd.read_csv("sales_data.csv")
print("Original Data:")
print(df)
```

```
Original Data:
  Customer Product   Sales  Quantity
0     John  Laptop  1000.0         1
1     Mary   Phone   500.0         2
2     John  Laptop  1000.0         1
3    David  Tablet   700.0         3
4      NaN   Phone     NaN         2
```

```
print("Missing values:")
print(df.isnull().sum())
```

```
Missing values:
Customer    1
Product     0
Sales       1
Quantity    0
dtype: int64
```

```
df["Customer"] = df["Customer"].fillna("Unknown")
df["Sales"] = df["Sales"].fillna(df["Sales"].mean())
print("Cleaned Data:")
print(df)
```

```
Cleaned Data:
  Customer Product   Sales  Quantity
0     John  Laptop  1000.0         1
1     Mary   Phone   500.0         2
2     John  Laptop  1000.0         1
3    David  Tablet   700.0         3
4  Unknown   Phone   800.0         2
```

```
df = df.drop_duplicates()
print("After removing duplicates:")
print(df)
```

```
After removing duplicates:
  Customer Product   Sales  Quantity
0     John  Laptop  1000.0         1
1     Mary   Phone   500.0         2
3    David  Tablet   700.0         3
4  Unknown   Phone   800.0         2
```

```
import sqlite3
connection = sqlite3.connect("sales_database.db")
df.to_sql( "sales",
    connection,
    if_exists="replace",
    index=False)
print("Data stored in SQL database successfully!")
```

```
Data stored in SQL database successfully!
```

```
query = """
SELECT Product, SUM(Sales) AS Total_Sales
FROM sales
GROUP BY Product
```

```
"""
report = pd.read_sql_query(query, connection)
print("Sales Report:")
print(report)
```

```
Sales Report:
   Product  Total_Sales
0  Laptop       1000.0
1   Phone       1300.0
2  Tablet        700.0
```

```
connection.close()
print("Pipeline completed successfully!")
```

```
Pipeline completed successfully!
```

2. ETL pipeline for an e-commerce dataset

```
import pandas as pd
customers = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "name": ["John", "Mary", "David"],
    "email": ["john@gmail.com", "mary@gmail.com", "david@gmail.com"]
})
products = pd.DataFrame({
    "product_id": [101, 102, 103],
    "product": ["Laptop", "Phone", "Tablet"],
    "price": [1000, 500, 700]
})
orders = pd.DataFrame({
    "order_id": [1001, 1002, 1003],
    "customer_id": [1, 2, 3],
    "product_id": [101, 102, 103],
    "quantity": [1, 2, 1]
})
payments = pd.DataFrame({
    "payment_id": [501, 502, 503],
    "order_id": [1001, 1002, 1003],
    "payment_status": ["Paid", "Paid", "Pending"]
})
print("Data extracted successfully!")
```

```
Data extracted successfully!
```

```
order_data = orders.merge(
    products,
```

```
    on="product_id"
)

order_data["total_amount"] = (
    order_data["price"] * order_data["quantity"])
print("Transformed Order Data:")
print(order_data)
```

```
Transformed Order Data:
   order_id  customer_id  product_id  quantity product  price  total_amount
0      1001            1         101         1  Laptop   1000          1000
1      1002            2         102         2   Phone    500          1000
2      1003            3         103         1  Tablet    700           700
```

```
order_data = order_data.merge(
    customers,
    on="customer_id")
print("Customer + Order Data:")
print(order_data)
```

```
Customer + Order Data:
   order_id  customer_id  product_id  quantity product  price  total_amount  \
0      1001            1         101         1  Laptop   1000          1000
1      1002            2         102         2   Phone    500          1000
2      1003            3         103         1  Tablet    700           700

    name           email
0   John   john@gmail.com
1   Mary   mary@gmail.com
2  David  david@gmail.com
```

```
order_data = order_data.merge(
    payments,
    on="order_id")
print("Complete E-commerce Data:")
print(order_data)
```

```
Complete E-commerce Data:
   order_id  customer_id  product_id  quantity product  price  total_amount  \
0      1001            1         101         1  Laptop   1000          1000
1      1002            2         102         2   Phone    500          1000
2      1003            3         103         1  Tablet    700           700

    name           email  payment_id payment_status
0   John   john@gmail.com         501           Paid
1   Mary   mary@gmail.com         502           Paid
2  David  david@gmail.com         503        Pending
```

```
import sqlite3
connection = sqlite3.connect("ecommerce.db")
customers.to_sql(
    "customers",
    connection,
    if_exists="replace",
    index=False
)
products.to_sql(
```

```
    "products",
    connection,
    if_exists="replace",
    index=False
)
orders.to_sql(
    "orders",
    connection,
    if_exists="replace",
    index=False
)
payments.to_sql(
    "payments",
    connection,
    if_exists="replace",
    index=False
)
order_data.to_sql(
    "order_report",
    connection,
    if_exists="replace",
    index=False
)
print("ETL pipeline completed!")
```

```
ETL pipeline completed!
```

```
query = """
SELECT product,
       SUM(quantity) AS total_quantity,
       SUM(total_amount) AS total_sales
FROM order_report
GROUP BY product
"""

report = pd.read_sql_query(query, connection)
print("E-commerce Sales Report:")
print(report)
```

```
E-commerce Sales Report:
   product  total_quantity  total_sales
0  Laptop               1         1000
1   Phone               2         1000
2  Tablet               1          700
```

```
connection.close()
```

3. Historical + newly arriving data pipeline

```
import pandas as pd
historical_data = pd.DataFrame({
    "order_id": [1, 2, 3],
    "customer": ["John", "Mary", "David"],
    "sales": [1000, 500, 700]})
historical_data
```

	order_id	customer	sales
0	1	John	1000
1	2	Mary	500
2	3	David	700

```
new_data = pd.DataFrame({
    "order_id": [4, 5],
    "customer": ["Alice", "Bob"],
    "sales": [800, 600]})
new_data
```

	order_id	customer	sales
0	4	Alice	800
1	5	Bob	600

```
combined_data = pd.concat(
    [historical_data, new_data],
    ignore_index=True
)

print("Combined Data:")
print(combined_data)
```

```
Combined Data:
   order_id customer  sales
0         1     John   1000
1         2     Mary    500
2         3    David    700
3         4    Alice    800
4         5      Bob    600
```

```
combined_data = combined_data.drop_duplicates(
    subset=["order_id"])
print("Data after removing duplicates:")
print(combined_data)
```

```
Data after removing duplicates:
   order_id customer  sales
0         1     John   1000
1         2     Mary    500
2         3    David    700
3         4    Alice    800
4         5      Bob    600
```

```
import sqlite3
connection = sqlite3.connect("historical_new_data.db")
combined_data.to_sql(
    "orders",
    connection,
    if_exists="replace",
    index=False)
print("Historical and new data loaded into database!")
```

```
Historical and new data loaded into database!
```

```
query = """
SELECT customer,
       SUM(sales) AS total_sales
FROM orders
GROUP BY customer
"""
final_report = pd.read_sql_query(
    query,
    connection
)
print("Final Report:")
print(final_report)
```

```
Final Report:
  customer  total_sales
0    Alice          800
1      Bob          600
2    David          700
3     John         1000
4     Mary          500
```

```
connection.close()
print("Historical + New Data Pipeline Completed!")
```

```
Historical + New Data Pipeline Completed!
```

Practical-10: End-to-End E-Commerce Data Engineering Pipeline
Aim

To design and implement an **end-to-end data engineering pipeline** for an e-commerce dataset by performing **data ingestion**, **data cleaning**, **data transformation**, **SQL database storage**, **data warehouse design**, **reporting**, **and visualization** using Python, Pandas, SQLite, and Matplotlib.

Code

```
# ============================================================
# END-TO-END E-COMMERCE DATA ENGINEERING PROJECT
# ============================================================
import pandas as pd
import sqlite3
import matplotlib.pyplot as plt
# ============================================================
# 1. DATA INGESTION
# ============================================================

# Customers
customers = pd.DataFrame({
    "customer_id": [1, 2, 3, 4, 5],
    "customer_name": ["John", "Mary", "David", "Alice", "Bob"],
    "city": ["Mumbai", "Delhi", "Pune", "Mumbai", "Chennai"]
})
# Products
products = pd.DataFrame({
    "product_id": [101, 102, 103, 104, 105],
    "product_name": [
        "Laptop",
        "Phone",
        "Tablet",
        "Headphones",
        "Keyboard"
    ],
    "category": [
        "Electronics",
        "Electronics",
        "Electronics",
        "Accessories",
        "Accessories"
    ],
    "price": [60000, 30000, 20000, 3000, 1500]
})

# Orders
orders = pd.DataFrame({
```

```
    "order_id": [1001, 1002, 1003, 1004, 1005],
    "customer_id": [1, 2, 3, 4, 5],
    "product_id": [101, 102, 103, 104, 105],
    "quantity": [1, 2, 1, 3, 2]
})

# Payments
payments = pd.DataFrame({
    "payment_id": [501, 502, 503, 504, 505],
    "order_id": [1001, 1002, 1003, 1004, 1005],
    "payment_status": [
        "Paid",
        "Paid",
        "Paid",
        "Pending",
        "Paid"
    ]
})


print("========== DATA INGESTION ==========")

print("\nCustomers:")
display(customers)

print("\nProducts:")
display(products)

print("\nOrders:")
display(orders)

print("\nPayments:")
display(payments)


# ============================================================
# 2. SAVE DATA AS CSV FILES
# ============================================================

customers.to_csv("customers.csv", index=False)
products.to_csv("products.csv", index=False)
orders.to_csv("orders.csv", index=False)
payments.to_csv("payments.csv", index=False)

print("\nAll CSV files created successfully!")

# ============================================================
# 3. DATA CLEANING
# ============================================================
```

```
print("\n========== DATA CLEANING ==========")
# Check missing values
print("\nMissing values in Customers:")
print(customers.isnull().sum())
print("\nMissing values in Products:")
print(products.isnull().sum())
print("\nMissing values in Orders:")
print(orders.isnull().sum())
print("\nMissing values in Payments:")
print(payments.isnull().sum())

# Remove duplicate records

customers = customers.drop_duplicates()
products = products.drop_duplicates()
orders = orders.drop_duplicates()
payments = payments.drop_duplicates()
print("\nDuplicates removed successfully!")


# Check data types
print("\nCustomers Data Types:")
print(customers.dtypes)
print("\nProducts Data Types:")
print(products.dtypes)

print("\nOrders Data Types:")
print(orders.dtypes)
print("\nPayments Data Types:")
print(payments.dtypes)


# Display cleaned datasets

print("\nCleaned Customers:")
display(customers)

print("\nCleaned Products:")
display(products)

print("\nCleaned Orders:")
display(orders)

print("\nCleaned Payments:")
display(payments)


# ============================================================
```

```
# 4. DATA TRANSFORMATION
# ============================================================

print("\n========== DATA TRANSFORMATION ==========")

# Join Orders and Products

order_data = orders.merge(
    products,
    on="product_id",
    how="inner"
)

print("\nOrders + Products:")
display(order_data)


# Calculate total sales

order_data["total_sales"] = (
    order_data["price"] *
    order_data["quantity"]
)

print("\nTotal Sales Calculated:")
display(order_data)


# Add customer information

order_data = order_data.merge(
    customers,
    on="customer_id",
    how="inner"
)

print("\nAdded Customer Information:")
display(order_data)


# Add payment information

order_data = order_data.merge(
    payments,
    on="order_id",
    how="inner"
)

print("\nFinal Transformed Dataset:")
```

```
display(order_data)


# ============================================================
# 5. SALES SUMMARY
# ============================================================

sales_summary = order_data.groupby("category").agg(
    total_sales=("total_sales", "sum"),
    total_quantity=("quantity", "sum")
).reset_index()

print("\nSales Summary by Category:")
display(sales_summary)


# ============================================================
# 6. CREATE SQL DATABASE
# ============================================================

print("\n========== SQL DATABASE STORAGE ==========")

connection = sqlite3.connect(
    "ecommerce_warehouse.db"
)

print("Database created successfully!")


# ============================================================
# 7. STORE TABLES IN DATABASE
# ============================================================

customers.to_sql(
    "dim_customer",
    connection,
    if_exists="replace",
    index=False
)
products.to_sql(
    "dim_product",
    connection,
    if_exists="replace",
    index=False
)
orders.to_sql(
    "orders",
    connection,
    if_exists="replace",
```

```
    index=False
)

payments.to_sql(
    "payments",
    connection,
    if_exists="replace",
    index=False
)
order_data.to_sql(
    "fact_sales",
    connection,
    if_exists="replace",
    index=False
)
print("All tables stored in the database!")
# ============================================================
# 8. DISPLAY DATABASE TABLES
# ============================================================
print("\nCustomer Dimension:")
display(
    pd.read_sql_query(
        "SELECT * FROM dim_customer",
        connection
    )
)
print("\nProduct Dimension:")
display(
    pd.read_sql_query(
        "SELECT * FROM dim_product",
        connection
    )
)
print("\nFact Sales:")
display(
    pd.read_sql_query(
        "SELECT * FROM fact_sales",
        connection
    )
)


# ============================================================
# 9. CHECK DATABASE TABLES
# ============================================================
tables = pd.read_sql_query(
    """
    SELECT name
    FROM sqlite_master
```

```
    WHERE type='table'
    """,
    connection
)
print("\nTables in Data Warehouse:")
display(tables)
# ============================================================
# 10. DATE DIMENSION
# ============================================================
date_data = pd.DataFrame({
    "date_id": [1, 2, 3, 4, 5],
    "date": [
        "2026-01-01",
        "2026-01-02",
        "2026-01-03",
        "2026-01-04",
        "2026-01-05"
    ],
    "year": [2026, 2026, 2026, 2026, 2026],
    "month": [1, 1, 1, 1, 1]
})
print("\nDate Dimension:")
display(date_data)
date_data.to_sql(
    "dim_date",
    connection,
    if_exists="replace",
    index=False
)
print("Date dimension created successfully!")
# ============================================================
# 11. REPORTING - TOTAL SALES
# ============================================================
print("\n========== REPORTING ==========")
query = """
SELECT SUM(total_sales) AS total_sales
FROM fact_sales
"""
total_sales_report = pd.read_sql_query(
    query,
    connection
)

print("\nTotal Sales:")
display(total_sales_report)
# ============================================================
# 12. SALES BY CATEGORY
# ============================================================
query = """
```

```
SELECT category,
       SUM(total_sales) AS total_sales
FROM fact_sales
GROUP BY category
"""
category_report = pd.read_sql_query(
    query,
    connection
)
print("\nSales by Category:")
display(category_report)
# ============================================================
# 13. SALES BY CUSTOMER
# ============================================================

query = """
SELECT customer_name,
       SUM(total_sales) AS total_sales
FROM fact_sales
GROUP BY customer_name
"""
customer_report = pd.read_sql_query(
    query,
    connection
)
print("\nSales by Customer:")
display(customer_report)
# ============================================================
# 14. PAYMENT REPORT
# ============================================================
query = """
SELECT payment_status,
       COUNT(*) AS number_of_orders,
       SUM(total_sales) AS total_sales
FROM fact_sales
GROUP BY payment_status
"""
payment_report = pd.read_sql_query(
    query,
    connection
)

print("\nPayment Report:")
display(payment_report)
# ============================================================
# 15. DATA VISUALIZATION
# ============================================================
plt.figure(figsize=(8, 5))
plt.bar(
```

```
    category_report["category"],
    category_report["total_sales"]
)
plt.title("Sales by Product Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.show()
# ============================================================
# 16. FINISH THE PIPELINE
# ============================================================
print("\n====================================")
print("END-TO-END DATA ENGINEERING PROJECT")
print("====================================")

print("✓ Data Ingestion")
print("✓ Data Cleaning")
print("✓ Data Transformation")
print("✓ SQL Storage")
print("✓ Data Warehouse Design")
print("✓ Reporting")
print("✓ Visualization")
print("\nPROJECT COMPLETED SUCCESSFULLY!")
# Close database connection
connection.close()
```